SOCIAL MEDIA IMPACT ON STUDENTS' LIFE

Complete Machine Learning Project Documentation

Project Detail	Information
Dataset	Social_media_impact_on_life.csv
Environment	Jupyter Notebook
Language	Python
Dataset size	4,500 rows × 16 columns initially
Target	Overall_Impact (Beneficial, Negative, Neutral)
Models	Logistic Regression, Random Forest, Gradient Boosting

Purpose of this document: Explain each project task, its code, purpose, reason, output, and the important concepts needed to understand the workflow and prepare for a viva.

1. Project Introduction

The project uses student-related information to predict the overall impact of social media. The dataset includes daily usage, sleep, stress, academic performance, and other attributes. The target column is **Overall_Impact**, with three classes: Beneficial, Negative, and Neutral.

Project workflow

• Import libraries and load the CSV dataset.

• Explore the data and inspect missing values, duplicates, and target classes.

• Perform exploratory data analysis (EDA) using graphs.

• Clean and preprocess the data.

• Train and compare three classification models.

• Use cross-validation to select the best model.

• Evaluate the selected model using test metrics and a confusion matrix.

2. Dataset Information

The original dataset contains 4,500 rows and 16 columns. The Student_ID column is an identifier, not a useful predictive feature, so it was removed before model training.

Column	Meaning
Student_ID	Unique student identifier
Age	Student's age
Gender	Gender category
Academic_Level	Student's academic level
Primary_Platform	Main social media platform
Daily_Usage_Hours	Daily social media usage
Weekend_Extra_Hours	Additional weekend usage
Device_Type	Device used
Sleep_Duration_Hours	Hours of sleep
Sleep_Quality_Score	Sleep quality score
Late_Night_Usage	Whether usage occurs late at night
Social_Comparison_Frequency	Frequency of social comparison
Perceived_Stress_Score	Perceived stress score
Mental_Health_Index	Mental health index
Academic_Performance_GPA	Academic GPA
Overall_Impact	Target class to predict

TASK 1 — Import Libraries

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
```

```
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.model_selection import StratifiedKFold
from sklearn.model_selection import cross_val_score

from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import GradientBoostingClassifier

from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import classification_report
from sklearn.metrics import ConfusionMatrixDisplay
```

Purpose: Import ready-made Python tools for data handling, visualization, preprocessing, model training, and evaluation.

• **Pandas:** Read and manage tabular data.

• **NumPy:** Numerical operations.

• **Matplotlib and Seaborn:** Graphs and visual analysis.

• **Scikit-learn:** Data splitting, preprocessing, model training, cross-validation, and metrics.

TASK 2 — Load Dataset

```
data_df = pd.read_csv("Social_media_impact_on_life.csv")
data_df.head()
```

Purpose: Read the CSV file into a Pandas DataFrame called **data_df**. The head() method displays the first five rows so we can inspect the records.

Expected output: A table showing the first five records.

TASK 3 — Explore Dataset

3.1 Dataset shape

```
data_df.shape
```

Output: (4500, 16) — 4,500 rows and 16 columns. This tells us the size of the original dataset.

3.2 Column names

```
data_df.columns
```

Purpose: Display all column names and identify the available features and target.

3.3 Dataset information

```
data_df.info()
```

Purpose: Check column data types, non-null counts, and memory use. Common types include int, float, object, and bool.

3.4 Numerical summary

```
data_df.describe()
```

Purpose: Show count, mean, standard deviation, minimum, quartiles, and maximum for numerical columns.

3.5 Missing values

```
data_df.isnull().sum()
```

Observed missing values: Perceived_Stress_Score = 46; Academic_Performance_GPA = 85. Total = 131.

Purpose: Find incomplete values before training.

3.6 Duplicate rows

```
data_df.duplicated().sum()
```

Output: 0. No duplicate rows were found.

3.7 Target distribution

```
data_df['Overall_Impact'].value_counts()
```

Class	Number of records
Beneficial	3,681
Neutral	654
Negative	165

The classes are imbalanced because Beneficial has many more records than Negative. This is why Macro F1-score is important in addition to accuracy.

TASK 4 — Exploratory Data Analysis (EDA)

Exploratory Data Analysis means examining data using statistics and visualizations before training a model. It helps reveal distributions, unusual values, and possible relationships.

4.1 Target distribution

```
plt.figure(figsize=(8, 5))
sns.countplot(data=data_df, x='Overall_Impact')
plt.title("Distribution of Overall Social Media Impact")
plt.xlabel("Overall Impact")
plt.ylabel("Number of Students")
plt.show()
```

Purpose: Compare the number of records in each target class.

4.2 Daily usage distribution

```
plt.figure(figsize=(8, 5))
sns.histplot(data_df['Daily_Usage_Hours'], kde=True)
plt.title("Daily Social Media Usage")
plt.xlabel("Daily Usage Hours")
plt.ylabel("Number of Students")
plt.show()
```

Purpose: Understand how daily usage hours are distributed.

4.3 Sleep duration distribution

```
plt.figure(figsize=(8, 5))
sns.histplot(data_df['Sleep_Duration_Hours'], kde=True)
plt.title("Sleep Duration Distribution")
plt.xlabel("Sleep Duration Hours")
plt.show()
```

Purpose: Inspect the distribution of students' sleep duration.

4.4 Correlation heatmap

```
plt.figure(figsize=(12, 8))
sns.heatmap(
    data_df.select_dtypes(include=np.number).corr(),
    annot=True,
    cmap='coolwarm'
)
```

```
plt.title("Feature Correlation Heatmap")
plt.show()
```

Purpose: Show linear correlations between numerical features. Positive correlation means two variables tend to move in the same direction; negative correlation means they tend to move in opposite directions. Correlation alone does not prove causation.

TASK 5 — Data Cleaning and Preprocessing

5.1 Fill missing numerical values with the median

```
data_df['Perceived_Stress_Score'] = data_df['Perceived_Stress_Score'].fillna(
    data_df['Perceived_Stress_Score'].median()
)

data_df['Academic_Performance_GPA'] = data_df['Academic_Performance_GPA'].fillna(
    data_df['Academic_Performance_GPA'].median()
)
```

Purpose: Replace missing values so that incomplete numerical entries do not interfere with model training. The median is the middle value after sorting and is less affected by extreme values than the mean.

5.2 Verify missing values

```
data_df.isnull().sum()
```

Observed result: The two previously incomplete columns had zero missing values after filling.

5.3 Remove Student_ID

```
data_df = data_df.drop(columns=['Student_ID'])
```

Purpose: Remove the identifier because it labels a student rather than describing a meaningful pattern for prediction.

5.4 Separate features and target

```
X = data_df.drop(columns=['Overall_Impact'])
y = data_df['Overall_Impact']
```

X: Input features used to make predictions. **y:** Target labels the model is learning to predict.

5.5 Separate numerical and categorical columns

```
numeric_columns = X.select_dtypes(
    include=np.number
).columns.tolist()

categorical_columns = X.select_dtypes(
    include=['object', 'bool', 'category']
).columns.tolist()
```

Observed result: 8 numerical columns and 6 categorical columns. We handle these two types differently because numerical values can be scaled, while categories need encoding.

5.6 Split training and testing data

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

Observed shapes: X_train = (3600, 14); X_test = (900, 14). The split uses 80% for training and 20% for testing.

• **test_size=0.20:** Reserve 20% for testing.

• **random_state=42:** Make the split reproducible.

• **stratify=y:** Preserve approximately the same class proportions in both portions.

5.7 Build preprocessing transformers

```
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer(transformers=[
    ('num', numeric_transformer, numeric_columns),
    ('cat', categorical_transformer, categorical_columns)
])
```

Numerical pipeline: Median imputation fills any missing numerical values; StandardScaler scales numerical features to comparable ranges.

Categorical pipeline: Most-frequent imputation fills missing categories; OneHotEncoder converts categories into numerical indicator columns. handle_unknown='ignore' prevents errors for categories not seen during fitting.

5.8 Apply preprocessing

```
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)
```

Observed shapes: Training = (3600, 32); Testing = (900, 32). The 14 original features became 32 encoded/scaled feature columns.

Important: Fit preprocessing on training data only, then use the learned transformations on test data. This prevents the test data from influencing preprocessing.

TASK 6 — Train Three Machine Learning Models

We trained three classification algorithms to compare their ability to predict Beneficial, Negative, and Neutral.

6.1 Logistic Regression

```
model_1 = LogisticRegression(
    max_iter=1000,
    class_weight='balanced',
    random_state=42
)
model_1.fit(X_train_processed, y_train)
```

Logistic Regression is a classification algorithm. max_iter=1000 allows more optimization iterations. class_weight='balanced' adjusts class weights to account for imbalance.

6.2 Random Forest

```
model_2 = RandomForestClassifier(
    n_estimators=200,
    class_weight='balanced',
    random_state=42
)
model_2.fit(X_train_processed, y_train)
```

Random Forest combines predictions from many decision trees. n_estimators=200 sets the number of trees to 200. Balanced class weights help account for class imbalance.

6.3 Gradient Boosting

```
model_3 = GradientBoostingClassifier(
    n_estimators=100,
    learning_rate=0.1,
    random_state=42
)
model_3.fit(X_train_processed, y_train)
```

Gradient Boosting builds a sequence of models, with later stages trying to improve on earlier errors. n_estimators=100 sets the number of boosting stages; learning_rate=0.1 controls each stage's contribution.

6.4 Generate test predictions

```
prediction_1 = model_1.predict(X_test_processed)
prediction_2 = model_2.predict(X_test_processed)
prediction_3 = model_3.predict(X_test_processed)
```

Purpose: Generate predicted target labels for the test records so the three models can be evaluated.

TASK 7 — Compare Model Performance

```
model_results = pd.DataFrame({
    'Model': [
        'Logistic Regression',
        'Random Forest',
        'Gradient Boosting'
    ],
    'Accuracy': [
        accuracy_score(y_test, prediction_1),
        accuracy_score(y_test, prediction_2),
        accuracy_score(y_test, prediction_3)
    ],
    'Precision': [
        precision_score(y_test, prediction_1, average='macro'),
        precision_score(y_test, prediction_2, average='macro'),
        precision_score(y_test, prediction_3, average='macro')
    ],
    'Recall': [
        recall_score(y_test, prediction_1, average='macro'),
        recall_score(y_test, prediction_2, average='macro'),
        recall_score(y_test, prediction_3, average='macro')
    ],
    'F1-Score': [
        f1_score(y_test, prediction_1, average='macro'),
        f1_score(y_test, prediction_2, average='macro'),
        f1_score(y_test, prediction_3, average='macro')
    ]
})
print(model_results)
```

Purpose: Compare Accuracy, Macro Precision, Macro Recall, and Macro F1 for the three models. Macro averaging calculates a metric for each class and gives each class equal weight.

• **Accuracy:** Fraction of all predictions that are correct.

• **Precision:** Of the records predicted as a class, how many truly belong to that class?

• **Recall:** Of the actual records in a class, how many were correctly identified?

• **F1-score:** Harmonic combination of precision and recall.

• **Macro average:** Gives each class equal importance, useful for imbalanced datasets.

TASK 8 — Initial Cross-Validation

```
cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

cv_scores = cross_val_score(
    best_model,
    X_train_processed,
    y_train,
    cv=cv,
    scoring='f1_macro'
)

print("CV Scores:", cv_scores)
print("Average CV Score:", cv_scores.mean() * 100)
```

Five-fold cross-validation divides the training data into five parts. Each fold is used once for validation while the other four are used for training.

Important limitation: This initial cross-validation used features that had been preprocessed before the folds were created. That can allow information from a validation fold to influence preprocessing. Task 9 fixes this by placing preprocessing inside each model pipeline.

TASK 9 — Pipeline-Based Cross-Validation and Model Selection

9.1 Create complete pipelines

```
pipeline_1 = Pipeline(steps=[
    ('preprocessing', preprocessor),
    ('model', LogisticRegression(
        max_iter=1000,
        class_weight='balanced',
        random_state=42
    ))
])

pipeline_2 = Pipeline(steps=[
    ('preprocessing', preprocessor),
    ('model', RandomForestClassifier(
        n_estimators=200,
        class_weight='balanced',
        random_state=42
    ))
])

pipeline_3 = Pipeline(steps=[
    ('preprocessing', preprocessor),
    ('model', GradientBoostingClassifier(
        n_estimators=100,
        learning_rate=0.1,
        random_state=42
    ))
])
```

Purpose: Combine preprocessing and model fitting. During cross-validation, each fold learns its own preprocessing rules from that fold's training portion only.

9.2 Evaluate pipelines with cross-validation

```
model_pipelines = {
    'Logistic Regression': pipeline_1,
    'Random Forest': pipeline_2,
    'Gradient Boosting': pipeline_3
}

cv_results = {}

for model_name, model_pipeline in model_pipelines.items():
    scores = cross_val_score(
        model_pipeline,
        X_train,
        y_train,
        cv=cv,
        scoring='f1_macro',
        n_jobs=-1
    )

    cv_results[model_name] = scores

    print(model_name)
    print("Scores:", scores)
    print("Average:", scores.mean() * 100)
```

Purpose: Compare all three models using the same five-fold Macro F1 evaluation on the original training features.

9.3 Create the comparison table

```
final_comparison = pd.DataFrame({
    'Model': list(cv_results.keys()),
```

```
    'Average CV F1 (%)': [
        scores.mean() * 100 for scores in cv_results.values()
    ],
    'Standard Deviation (%)': [
        scores.std() * 100 for scores in cv_results.values()
    ]
}).sort_values(
    by='Average CV F1 (%)',
    ascending=False
).reset_index(drop=True)

print(final_comparison.to_string(index=False))
```

Purpose: Display each model's average cross-validation Macro F1 and the variation between folds. A higher average is better; a smaller standard deviation can indicate more consistent fold results.

9.4 Select the best pipeline

```
best_model_name = final_comparison.iloc[0]['Model']
best_pipeline = model_pipelines[best_model_name]

print("Selected Model:", best_model_name)
print(
    "Average CV Macro F1:",
    round(final_comparison.iloc[0]['Average CV F1 (%)'], 2),
    "%"
)
```

Observed output: Selected Model = Gradient Boosting; Average CV Macro F1 = 94.37%. It was selected because it achieved the highest average cross-validation Macro F1 among the three tested pipelines.

TASK 10 — Train the Final Model

```
best_pipeline.fit(X_train, y_train)
print("Final model training completed successfully!")
```

Observed output: Final model training completed successfully!

Purpose: Fit the selected complete pipeline on all training records. The pipeline performs preprocessing and model fitting together.

TASK 11 — Final Model Evaluation

```
final_predictions = best_pipeline.predict(X_test)

final_accuracy = accuracy_score(y_test, final_predictions)
final_f1 = f1_score(y_test, final_predictions, average='macro')

print("Final Model Performance")
print("-----------------------")
print("Accuracy:", round(final_accuracy * 100, 2), "%")
print("Macro F1-Score:", round(final_f1 * 100, 2), "%")
```

Metric	Observed result
Accuracy	97.33%
Macro F1-score	93.53%

Accuracy measures the overall fraction of correct predictions. Macro F1 averages class-level F1 scores equally, so it is especially informative when class counts are uneven.

TASK 12 — Classification Report

```
print("Classification Report")
print("---------------------")

print(classification_report(
    y_test,
    final_predictions,
```

```
    zero_division=0
))
```

Class	Precision	Recall	F1-score	Support
Beneficial	0.99	0.99	0.99	736
Negative	0.91	0.91	0.91	33
Neutral	0.90	0.92	0.91	131
Macro avg	0.93	0.94	0.94	900
Weighted avg	0.97	0.97	0.97	900

Precision: How many predictions for a class were correct. **Recall:** How many actual records of a class were found. **F1-score:** Combines precision and recall. **Support:** Number of actual test records in that class.

Beneficial has 736 test records, Negative has 33, and Neutral has 131. The Macro Average treats the three classes equally; the Weighted Average gives more influence to classes with more records.

TASK 13 — Confusion Matrix

```
from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt

ConfusionMatrixDisplay.from_predictions(
    y_test,
    final_predictions,
    cmap='Blues'
)

plt.title("Gradient Boosting - Confusion Matrix")
plt.show()
```

The confusion matrix compares actual classes (rows) with predicted classes (columns). The diagonal contains correct predictions; off-diagonal values are incorrect predictions.

Actual class	Predicted Beneficial	Predicted Negative	Predicted Neutral
Beneficial	726	0	10
Negative	0	30	3
Neutral	8	3	120

Correct predictions: 726 + 30 + 120 = 876.

Incorrect predictions: 10 + 3 + 8 + 3 = 24.

Total test records: 876 + 24 = 900. This agrees with the reported accuracy of 97.33% (876/900, rounded).

Final Project Conclusion

We analyzed a dataset about social media's impact on students using Python and Machine Learning. We explored the dataset, handled missing numerical values, removed the Student_ID identifier, separated input features from the target, encoded categorical variables, scaled numerical variables, and split the data into training and testing portions.

We trained Logistic Regression, Random Forest, and Gradient Boosting classifiers. We then used pipeline-based five-fold cross-validation to compare the models. Gradient Boosting achieved the highest average cross-validation Macro F1-score of 94.37%, so it was selected as the best model.

Final result	Value
Selected model	Gradient Boosting

Final result	Value
Average CV Macro F1	94.37%
Test accuracy	97.33%
Test Macro F1	93.53%
Correct test predictions	876 / 900
Incorrect test predictions	24 / 900

The model performed well on the supplied dataset when classifying records as Beneficial, Negative, or Neutral. These results describe predictive patterns in this dataset; they do not prove that social media causes a particular outcome.

Important Technical Notes

• The initial three-model comparison used the test dataset before the final cross-validation-based selection. Therefore, the reported test performance is not a completely untouched final holdout estimate. For a strictly independent evaluation, reserve a fresh test set before model selection.

• Pipeline-based cross-validation in Task 9 is the more appropriate cross-validation approach because preprocessing is fitted separately within each fold.

• The original missing values were filled before the train-test split in the notebook workflow. For a fully leakage-safe workflow, missing-value imputation should also be learned only from the training portion, inside the pipeline. The pipeline already includes imputers, so a future revision can preserve the raw missing values until after splitting.

• Check how the dataset's Overall_Impact target was created. If it was calculated directly from some of the input features, the model may be learning that construction rule rather than discovering an independent real-world relationship.

Viva Preparation — Important Questions

Question	Simple answer
What is Machine Learning?	A way for computers to learn patterns from data.
What is classification?	Predicting a category or class.
What are features?	Input columns used to make predictions.
What is the target?	The output column we want to predict.
What is preprocessing?	Preparing data before model training.
What is a missing value?	A value that is not present in a record.
Why use the median?	It is less affected by extreme values than the mean.
What is encoding?	Converting categories into numerical representation.
What is scaling?	Putting numerical features on comparable scales.
What is train-test split?	Separating data for learning and evaluation.
What is cross-validation?	Evaluating a model over multiple data splits.
What is Logistic Regression?	A classification algorithm.
What is Random Forest?	An ensemble of decision trees.
What is Gradient Boosting?	A sequential boosting method that improves predictions in stages.

Question	Simple answer
What is accuracy?	The proportion of all predictions that are correct.
What is precision?	The proportion of predicted class records that are correct.
What is recall?	The proportion of actual class records correctly identified.
What is F1-score?	A combined measure of precision and recall.
What is a confusion matrix?	A table comparing actual and predicted classes.
Why select Gradient Boosting?	It had the highest average cross-validation Macro F1 among the three models.